

Γεώργιος Κυριάκος
03119204
2η Εργαστηριακή Άσκηση ΨΕΣ

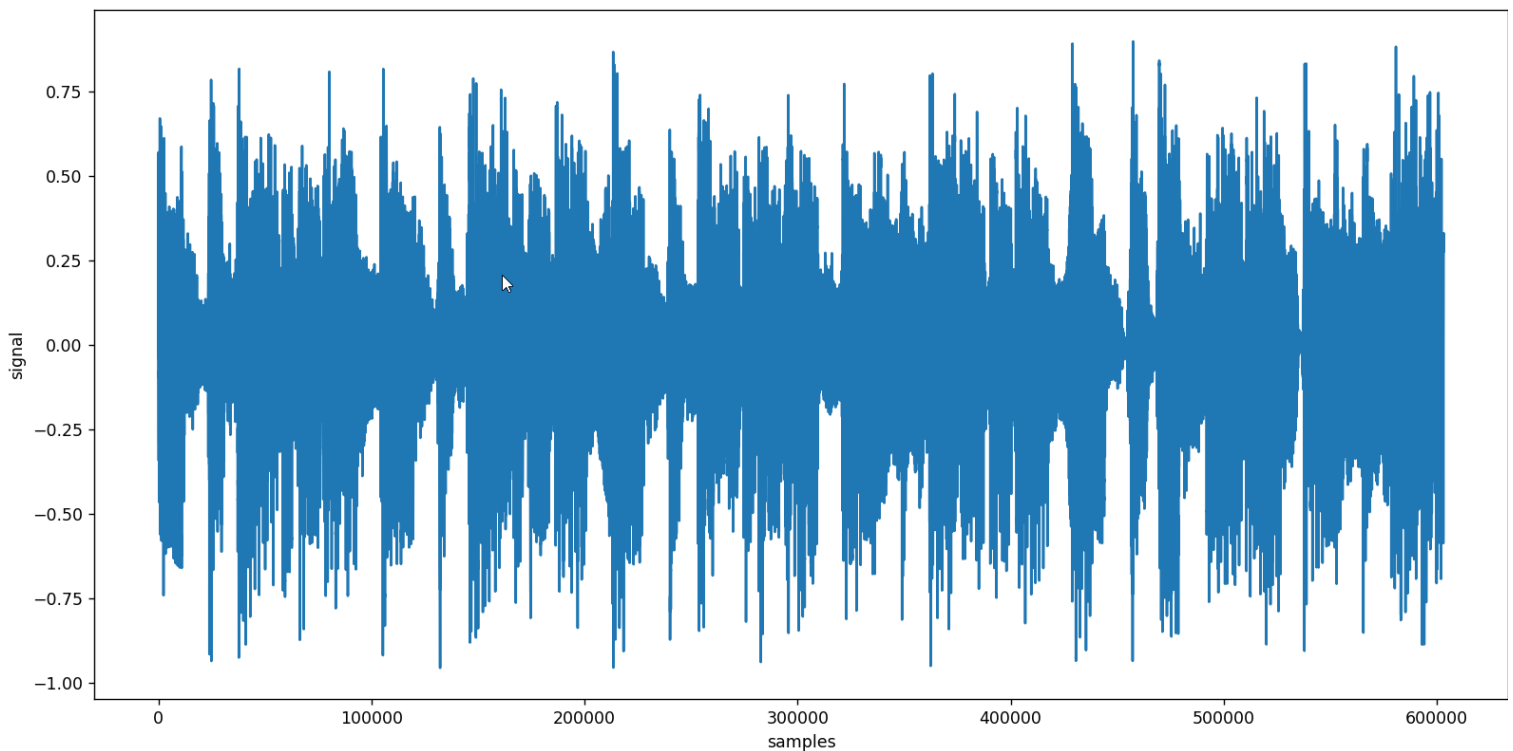
Μέρος 1

Βήμα 1.0: Προεπεξεργασία του σήματος

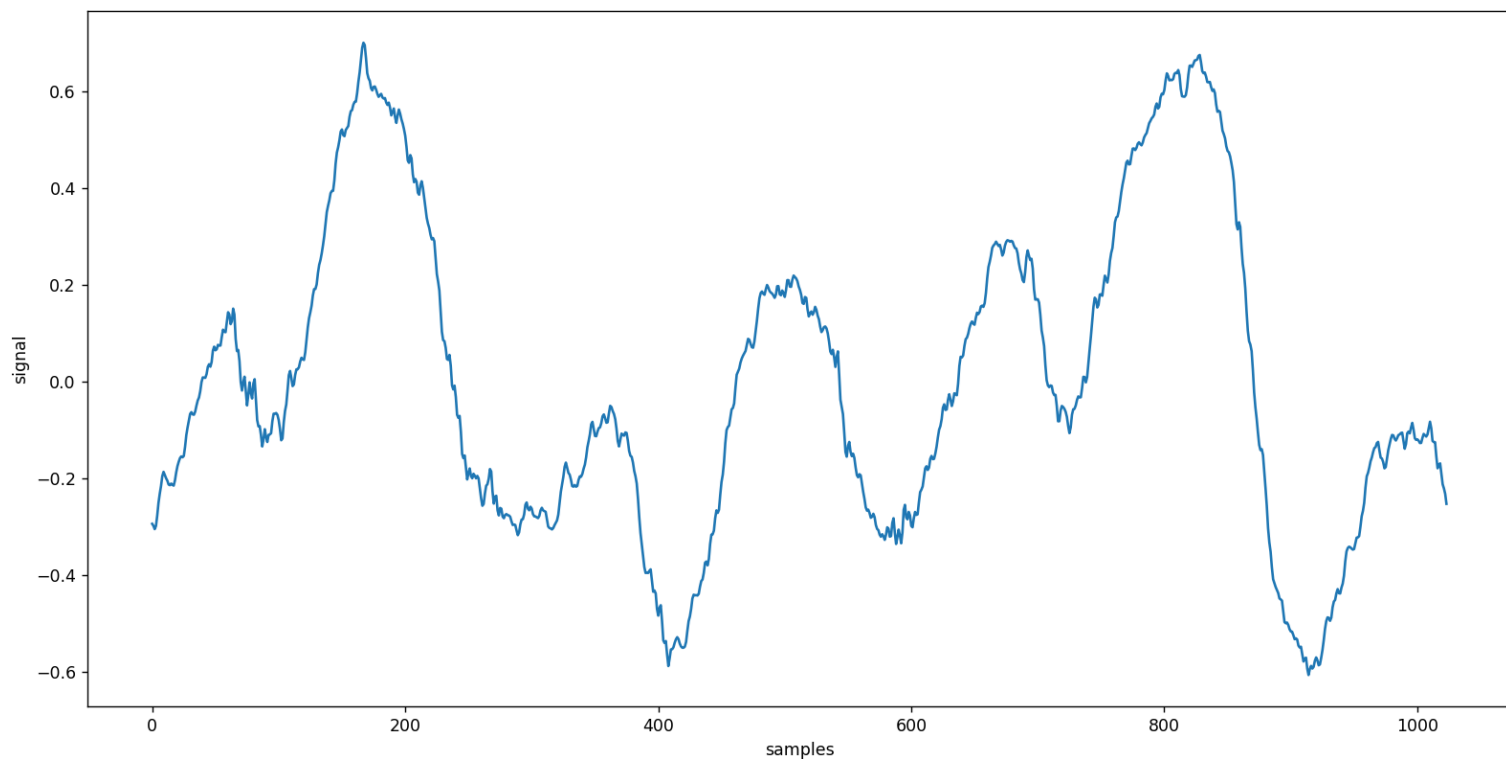
Αρχικά, μετατρέπω το σήμα από stereo σε mono χρησιμοποιώντας το μέσο όρο των δύο καναλιών. Το σήμα φαίνεται στο σχήμα 1.1. Όπως είναι φανερό είναι δύσκολο να καταλάβουμε τη μορφή του σήματος λόγω του μεγάλου αριθμού των δειγμάτων.

Για τον λόγο αυτό, απεικονίζω το κανονικοποιημένο σήμα (σχήμα 1.2) σε διάστημα δύο παραθύρων (κάθε παράθυρο έχει εύρος 512 δείγματα).

Από το σχήμα 1.2 φαίνεται ότι πρόκειται για ένα περιοδικό σήμα ημιτονικής μορφής



Σχήμα 1.1



Σχήμα 1.2

Όσον αφορά στην παραθυροποίηση του σήματος, το αρχικό μέγεθος του μετρήθηκε (με χρήση της `np.size()`) ως 603285 δείγματα. Όμως, το μέγεθος αυτό δεν είναι πολ/σιο του ζητούμενου μεγέθους του παραθύρου (512 δείγματα). Για τον λόγο αυτό προσθέτω μηδενικά στο τέλος του σήματος (με την `np.pad()`) και τελικά χωρίζω το σήμα σε 1179 πλαίσια ανάλυσης.

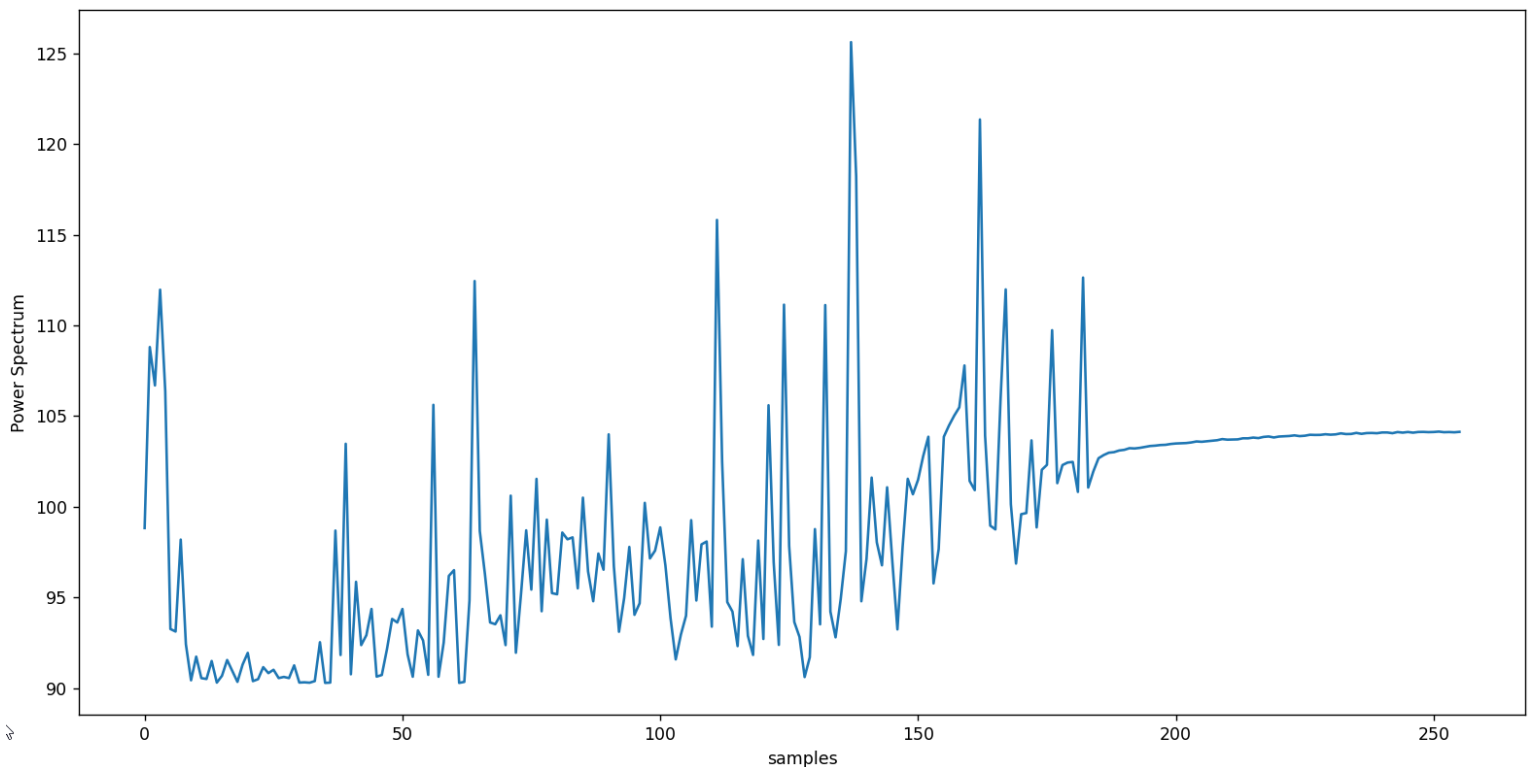
Βήμα 1.1: Φασματική Ανάλυση

Ακολουθώ τα βήματα της εκφώνησης και κατασκευάζω το φάσμα ισχύος το οποίο φαίνεται στο σχήμα 1.3 για ένα τυχαίο πλαίσιο ανάλυσης. Σημειώνεται ότι κάθε παράθυρο έχει πλέον 256 δείγματα λόγω της συμμετρίας που αναφέρεται και στην εκφώνηση.

Για να βρω το φάσμα ισχύος έχω πρώτα πολλαπλασιάσει με το hamming window κάθε παράθυρο του σήματος μουσικής.

Η διαδικασία υπολογισμού είναι η εξής :

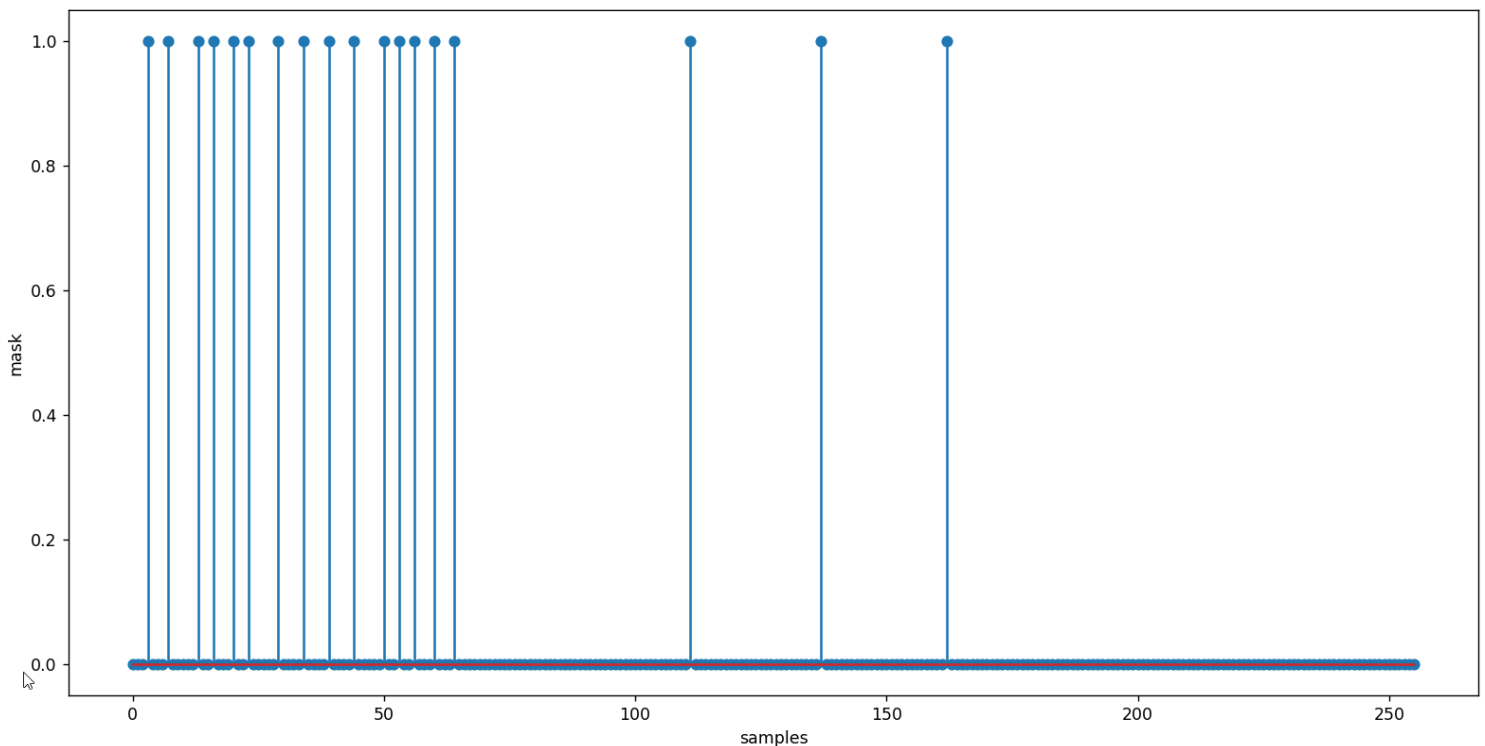
1. Απομόνωση ενός παραθυροποιημένου πλαισίου του κανονικοποιημένου σήματος μουσικής.
2. Εύρεση του τετραγώνου του μέτρου του.
3. Εύρεση του DFT
4. Μετατροπή σε dB και άθροιση με τη σταθερά PN



Σχήμα 1.3

Βήμα 1.2: Εντοπισμός масκών τόνων και θορύβου (Maskers)

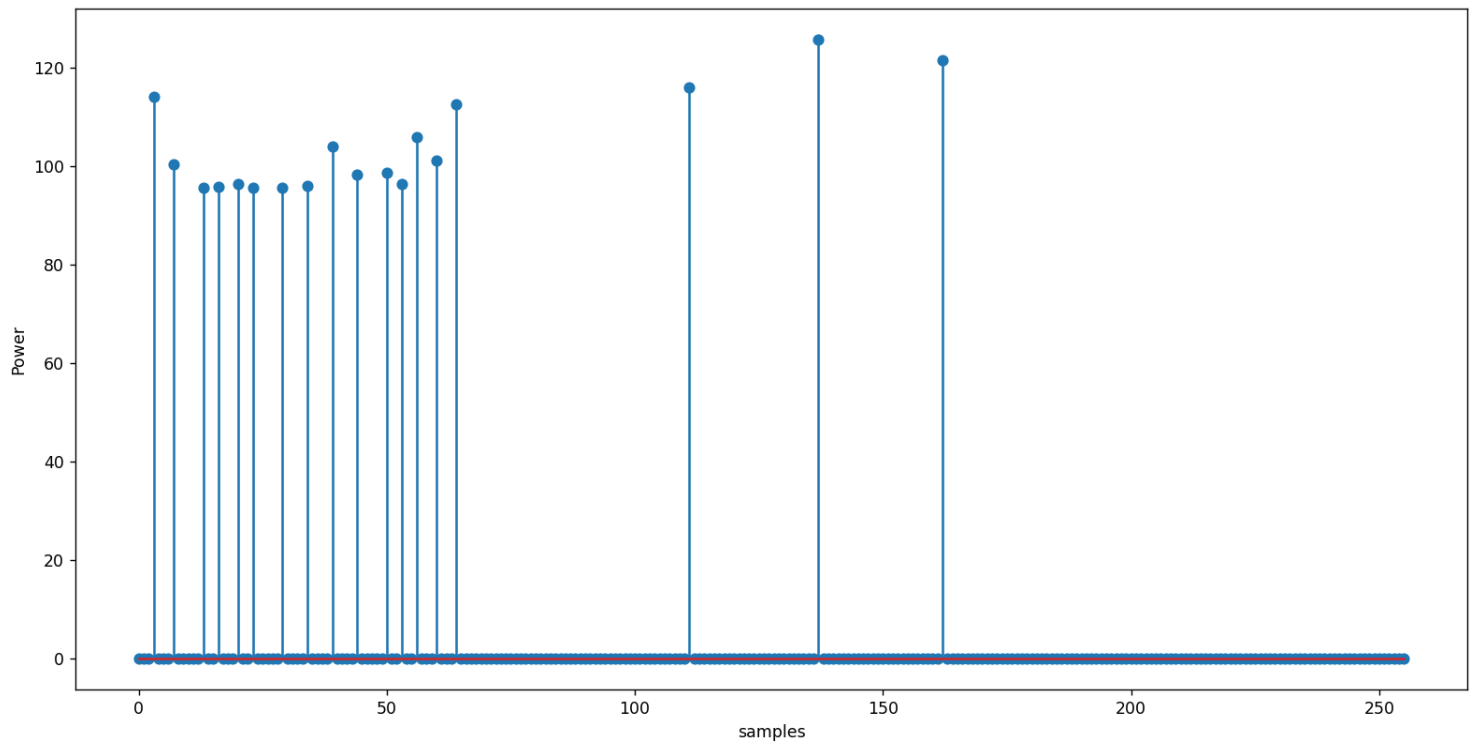
Κατασκευάζω τον πίνακα $St[k]$ (αναφέρεται ως `St_array` στον κώδικα), ο οποίος αποθηκεύει την τιμή 1 σε κάθε συχνότητα η οποία αποτελεί μάσκα για τις γειτονικές της, και 0 διαφορετικά (σύμφωνα με τη δοσμένη συνάρτηση). Στο σχήμα 1.4 φαίνεται ότι το πλήθος των масκών στο συγκεκριμένο (τυχαία επιλεγμένο παράθυρο) είναι 18 (πλήθος κορυφών), το οποίο επιβεβαιώθηκε και με τη χρήση της `np.where()`.



Σχήμα 1.4

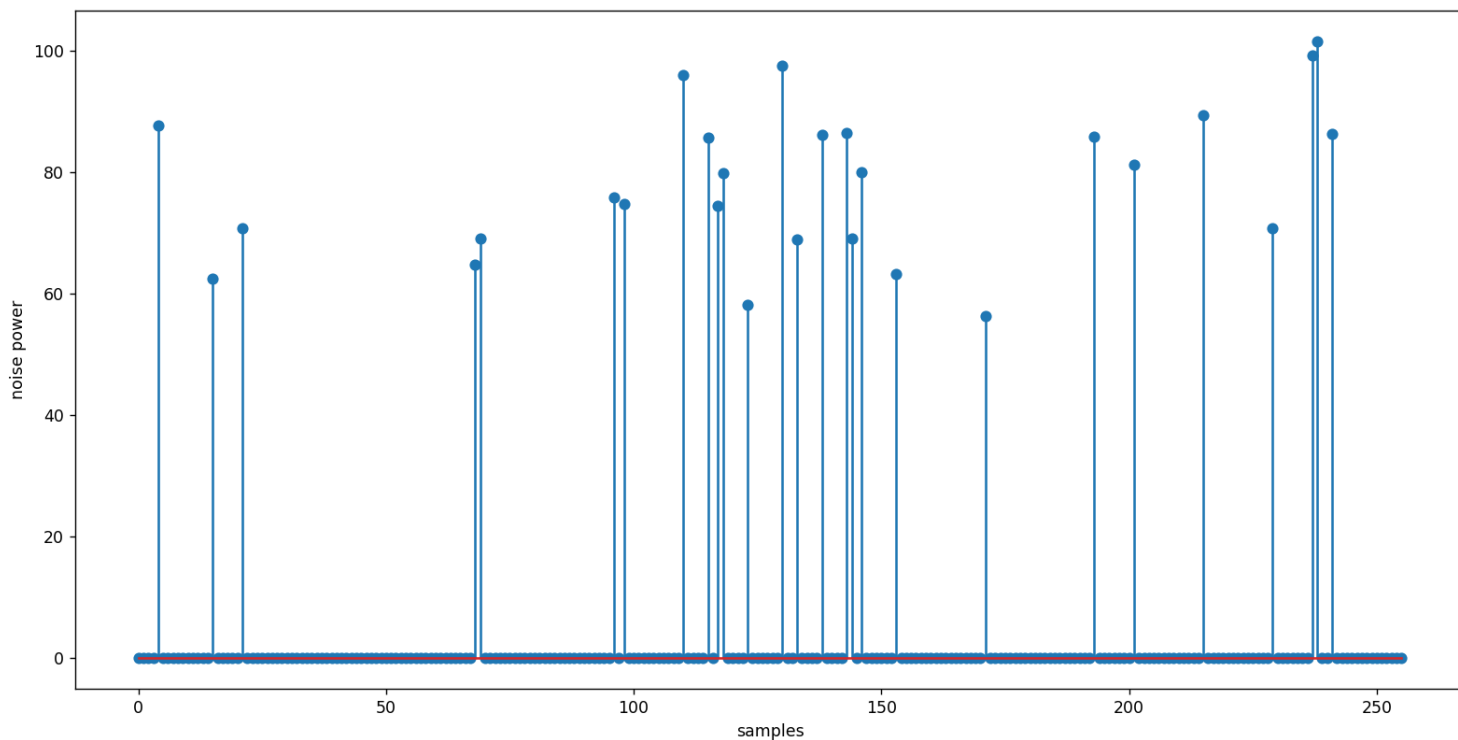
Μάλιστα, με χρήση της `np.where()` υπολόγισα τον μέσο αριθμό масκών για κάθε παράθυρο. Βάσει του υπολογισμού, ο μέσος αυτός όρος είναι 14-15 μάσκες. Επιπροσθέτως, υπολόγισα την διασπορά η οποία είναι αρκετά μικρή. Άρα, δεν αναμένω μεγάλες αποκλίσεις σε κάποια παράθυρα.

Αφού υπολόγισα την θέση των τονικών масκών είναι πλέον εύκολος και ο υπολογισμός της ισχύος του, η οποία απεικονίζεται στο σχήμα 1.5. Όπως ήταν αναμενόμενο οι θέσεις που η ισχύς είναι μη μηδενική ταυτίζεται με τις θέσεις ύπαρξης των масκών.



Σχήμα 1.5

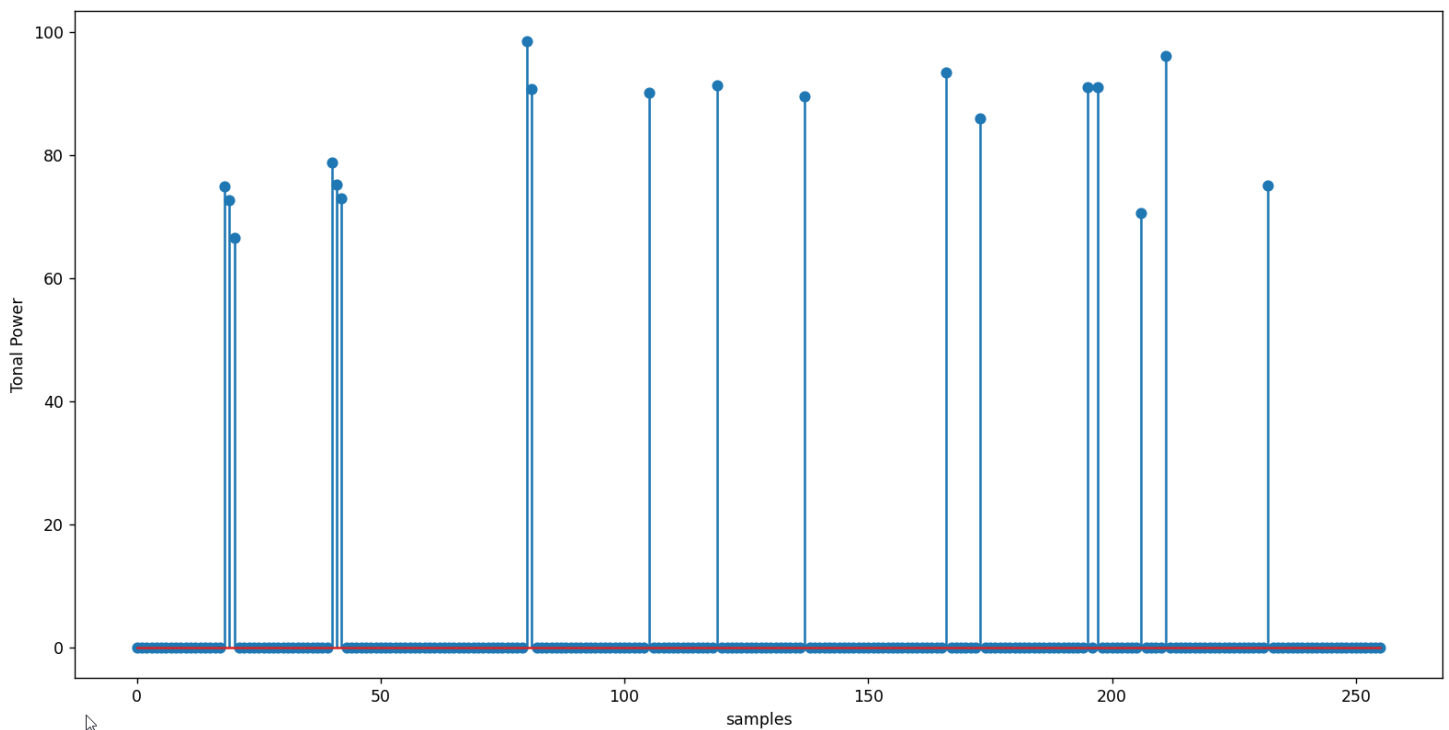
Εν συνεχεία φορτώνω την ισχύ θορύβου η οποία απεικονίζεται στο σχήμα 1.6 για το ίδιο πλαίσιο ανάλυσης. Με χρήση της `np.where()` (ή ισοδύναμα μετρώντας τις κορυφές) βρίσκω ότι ο αριθμός των μασκών θορύβου είναι 27. Αντίστοιχα με πριν ο μέσος αριθμός μασκών θορύβου σε κάθε παράθυρο είναι 22.



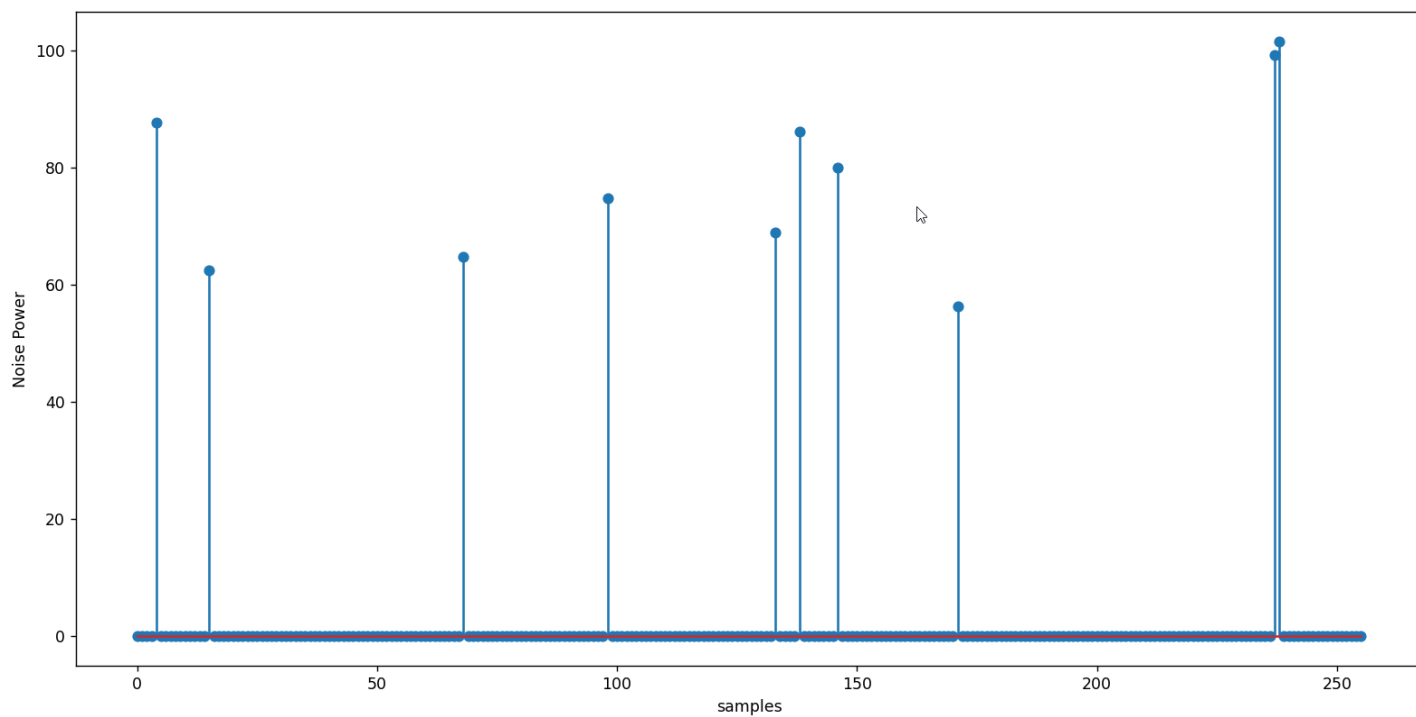
Σχήμα 1.6

Βήμα 1.3: Μείωση και αναδιοργάνωση των μασκών

Σε αυτό το βήμα δίνονται οι αναδιοργανωμένες μάσκες τόνων και θορύβου. Για το παραπάνω πλαίσιο ανάλυσης παρατηρείται αναδιοργάνωση των μασκών τόνων (ύπαρξη σε άλλες θέσεις). Όσον αφορά τις μάσκες θορύβου παρατηρήθηκε και μείωση (από 27 σε 10) και αναδιοργάνωση. Οι καινούργιες μάσκες τόνων και θορύβου απεικονίζονται στα σχήματα 1.7 και 1.8 αντίστοιχα.



Σχήμα 1.7



Σχήμα 1.8

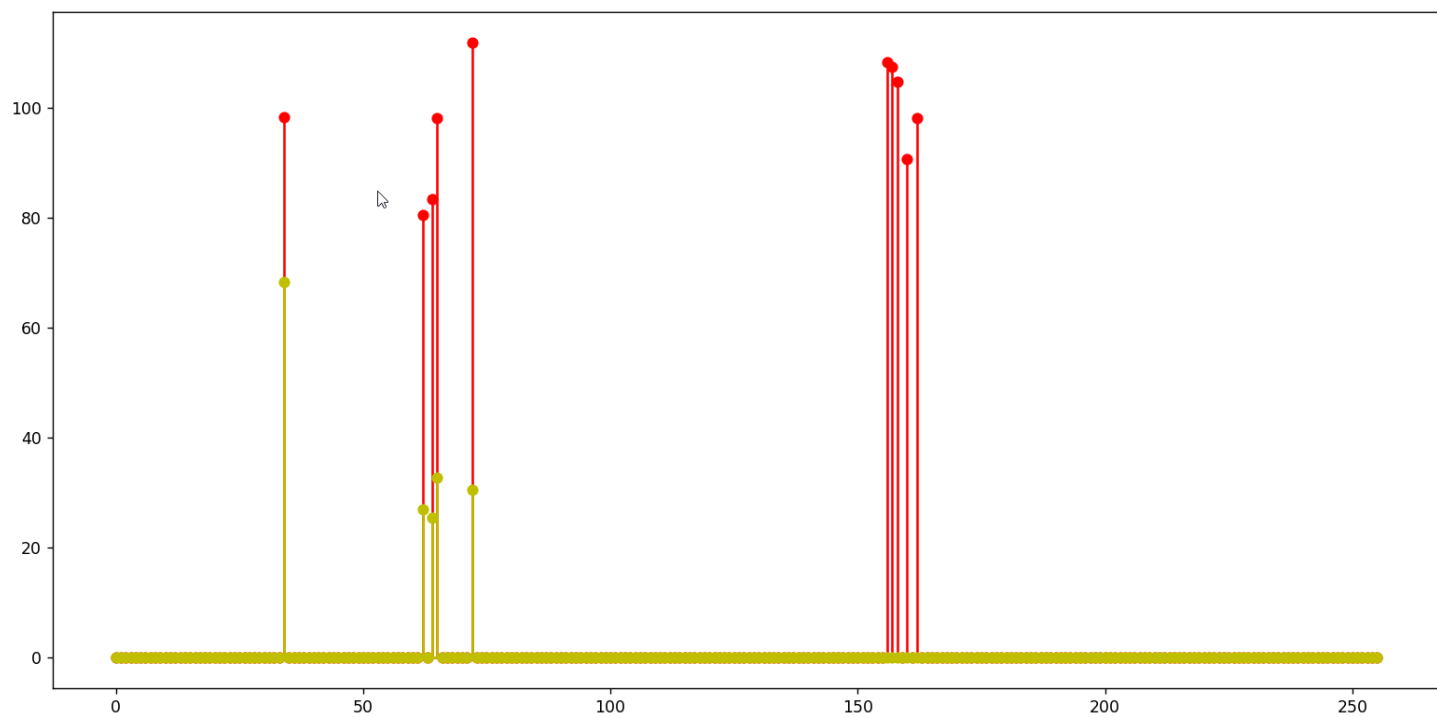
Βήμα 1.4: Υπολογισμός των δύο διαφορετικών κατωφλίων κάλυψης (Individual Masking Thresholds)

Για τον υπολογισμό των κατωφλίων κάλυψης κατασκευάζω αρχικά την συνάρτηση S_f για τις μάσκες τόνων και θορύβου σύμφωνα με την εκφώνηση της άσκησης. Για τον υπολογισμό των T_{tm} , T_{th} παρατηρώ ότι με ενδιαφέρουν μόνο οι θέσεις στις οποίες έχω $P > 0$. Έτσι, χρησιμοποιώντας την `nr.where()`, απομονώνω αυτές τις θέσεις και στη συνέχεια υπολογίζω τα Δb . Για τα Δb που είναι στην επιτρεπτή γειτονιά υπολογίζω τα κατώφλια κάλυψης. Άρα, στις θέσεις στις οποίες το P είναι μηδενικό ή δεν ανήκουν στην ζητούμενη γειτονιά των 12 bark το αντίστοιχο κατώφλι κάλυψης είναι μηδενικό.

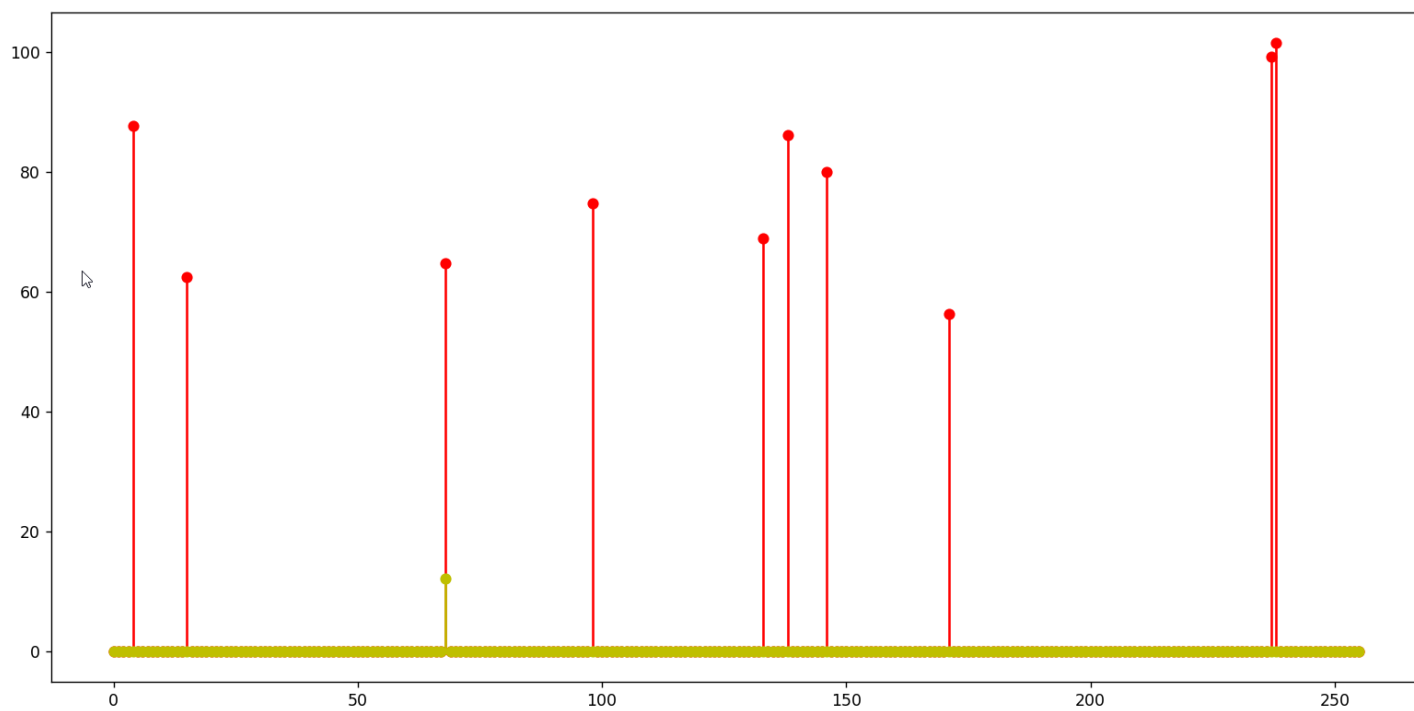
Τα παραπάνω φαίνονται εύκολα από τα επόμενα σχήματα.

Στο σχήμα 1.9 απεικονίζεται με κόκκινο χρώμα η μάσκα τόνων (P_{tm}) για το παράθυρο 50 και με κίτρινο το κατώφλι κάλυψης (T_{tm}) για το παράθυρο 50 στο δείγμα 50. Όπως είναι εμφανές οι μη μηδενικές θέσεις του T_{tm} είναι και μη μηδενικές θέσεις του P_{tm} . Ωστόσο, υπάρχουν θέσεις στις οποίες $P_{tm} > 0$ και $T_{tm} = 0$ καθώς δεν ανήκουν στην γειτονιά των 12 bark για το δείγμα 50.

Αντίστοιχα συμπεράσματα προκύπτουν και για τις μάσκες θορύβου στο σχήμα 1.10 (παράθυρο 100, δείγμα 50).



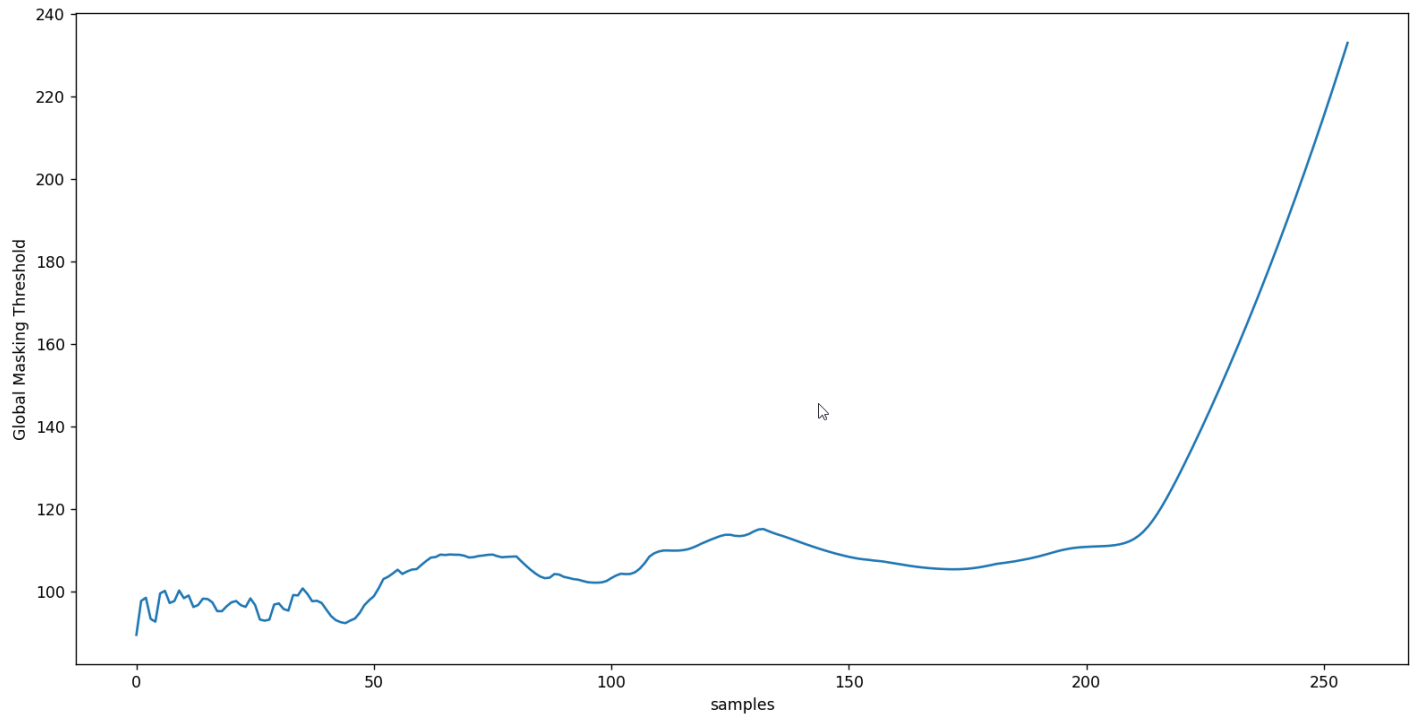
Σχήμα 1.9



Σχήμα 1.10

Βήμα 1.5: Υπολογισμός του συνολικού κατωφλίου κάλυψης (Global Masking Threshold)

Χρησιμοποιώντας τον τύπο της εκφώνησης υπολογίζω το T_g . Στο σχήμα 1.11 φαίνεται το T_g για κάποιο πλαίσιο ανάλυσης.

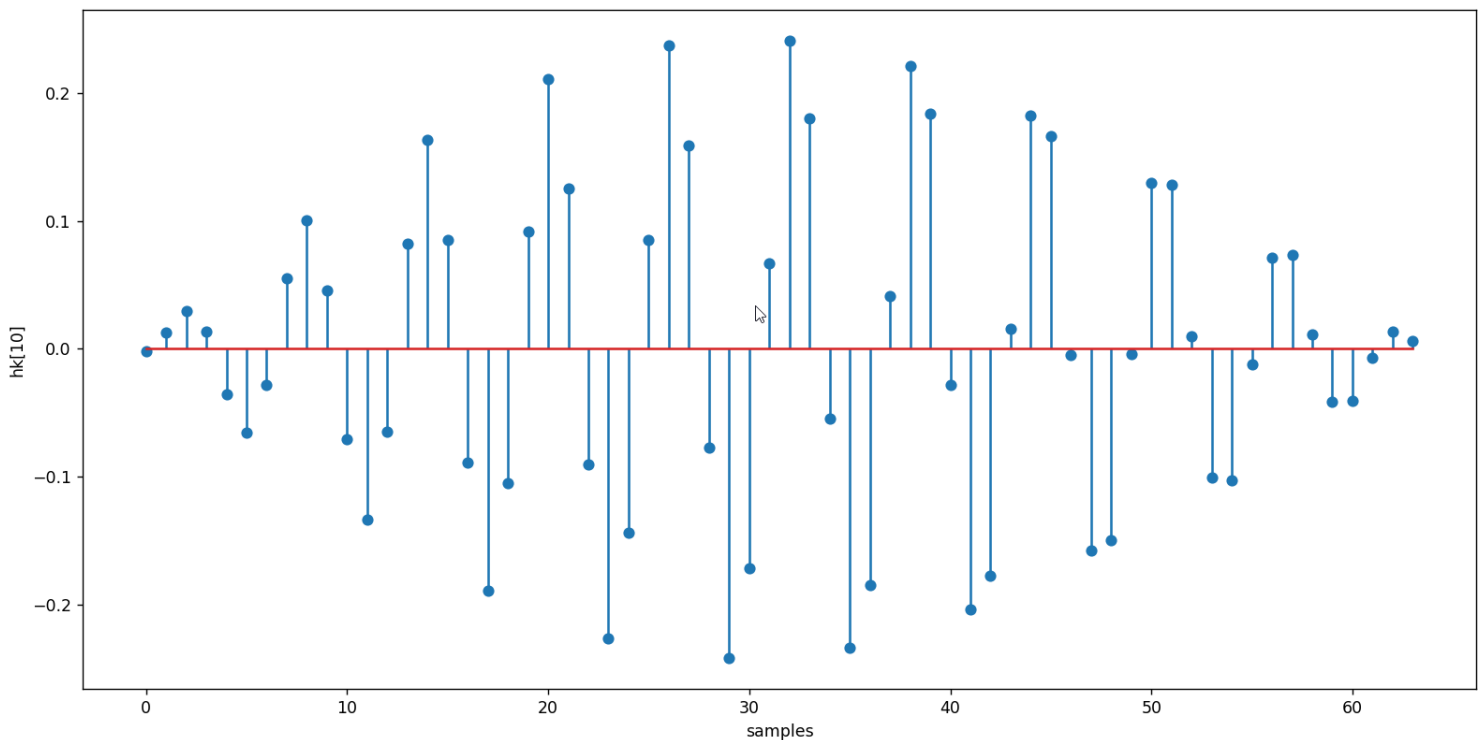


Σχήμα 1.11

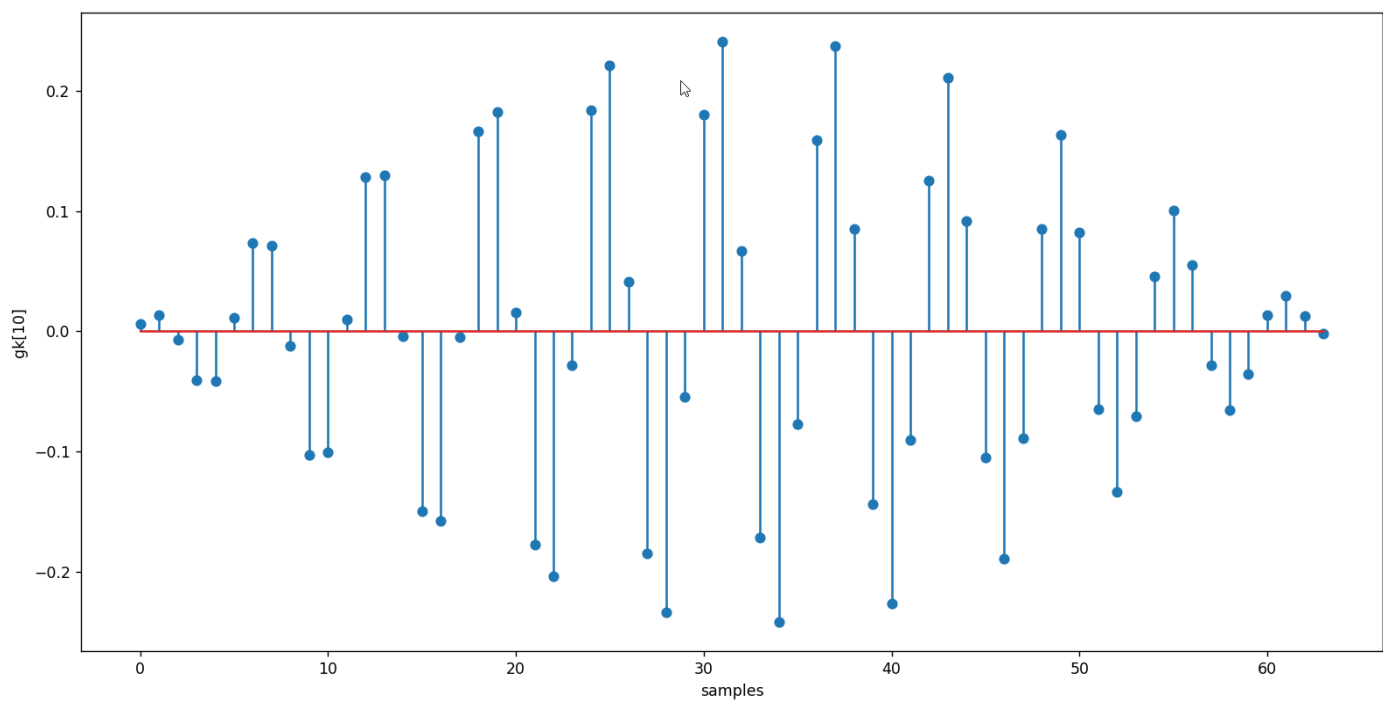
Μέρος 2

Βήμα 2.0: Συστοιχία Ζωνοπερατών Φίλτρων (Filterbank)

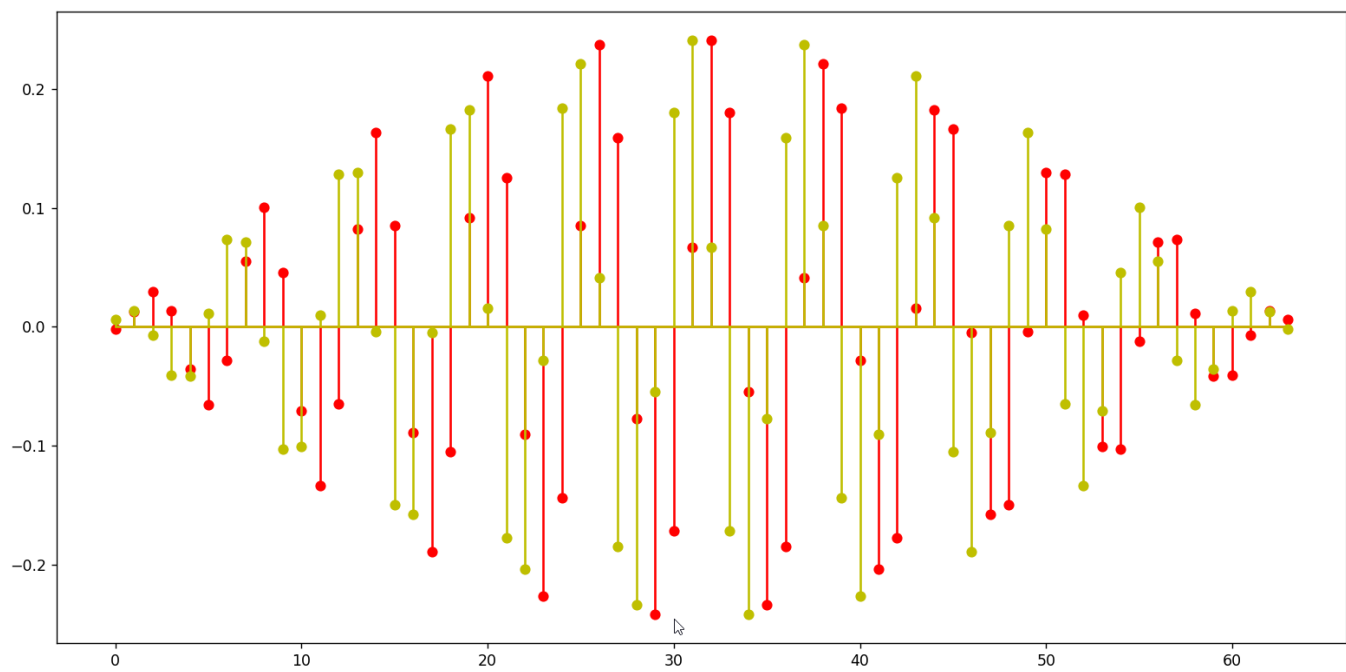
Με χρήση των συναρτήσεων $\sin$ και $\cos$ της numpy κατασκευάζω τα ζητούμενα φίλτρα. Στα σχήματα 2.1 και 2.2 απεικονίζονται τα φίλτρα αυτά για $k = 10$. Επίσης, στο σχήμα 2.3 απεικονίζονται και τα δύο φίλτρα (με κόκκινο το h_k) για να φανεί η διαφορά φάσης και η χρονική αντιστροφή.



Σχήμα 2.1



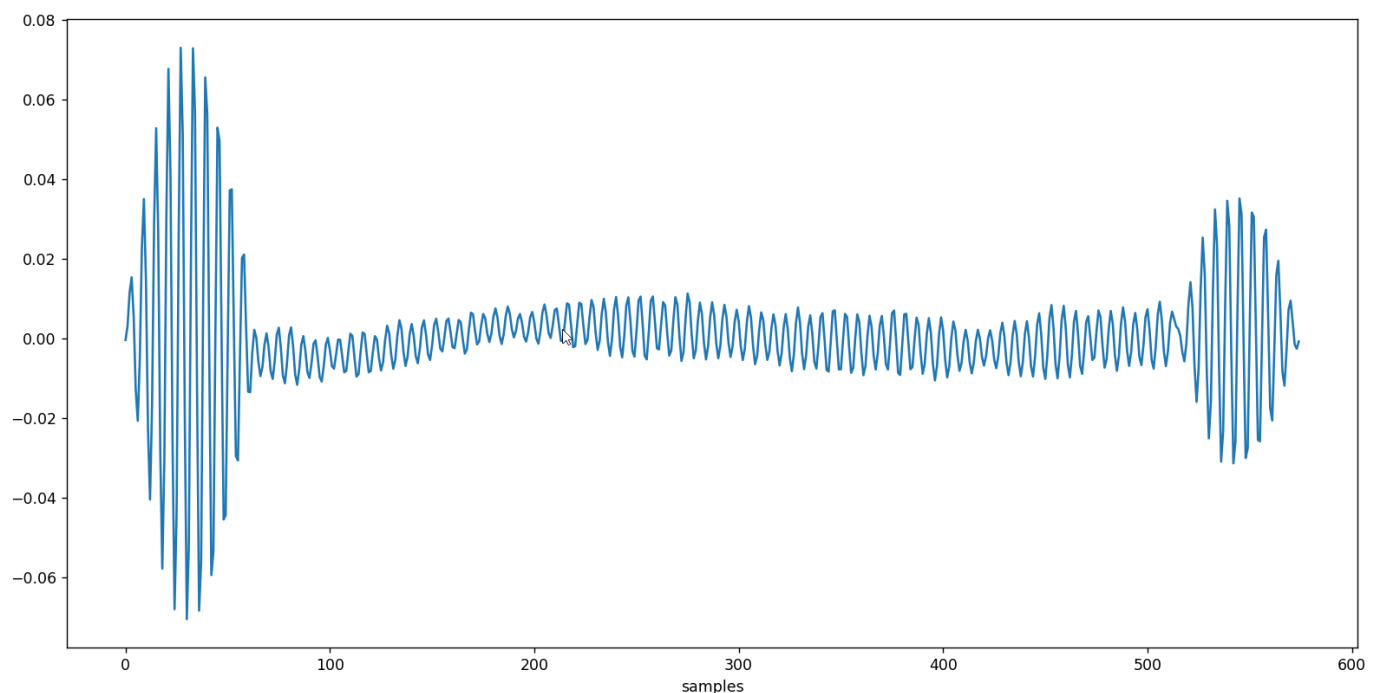
Σχήμα 2.2



Σχήμα 2.3

Βήμα 2.1: Ανάλυση με Συστοιχία Φίλτρων

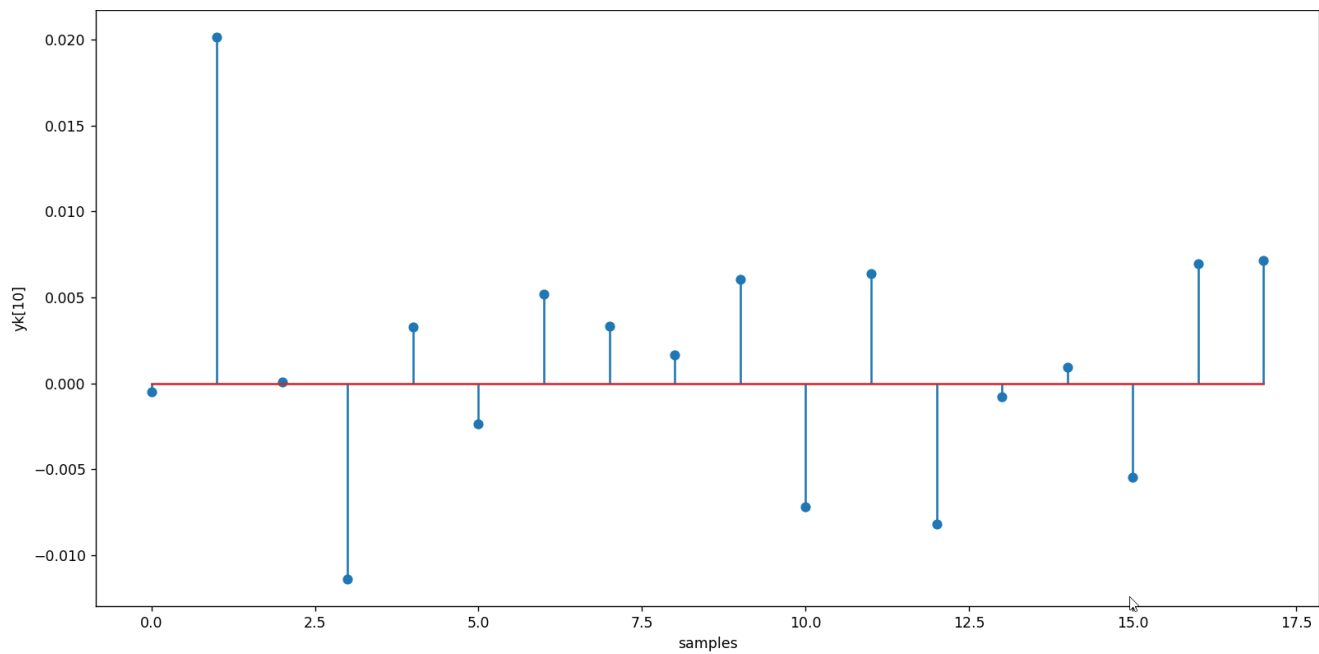
Αρχικά, χρησιμοποιώντας την `np.convolve()` υλοποιώ τα u_k ως συνέλιξη των h_k με το σήμα μουσικής (η συνέλιξη πραγματοποιείται σε επίπεδο frame, δηλαδή συνελίσσεται το h_k με το εκάστοτε πλαίσιο ανάλυσης). Επειδή κάθε πλαίσιο ανάλυσης αποτελείται από 512 δείγματα και κάθε φίλτρο από 64, θα προκύψει ένα σήμα του οποίου κάθε frame αποτελείται από 575 δείγματα ($512 + 64 - 1$). Στο σχήμα 2.4 απεικονίζεται ένα πλαίσιο (575 δειγμάτων) του u_{10} .



Σχήμα 2.4

Ύστερα, πραγματοποιείται το downsampling. Επειδή το downsampling γίνεται με παράγοντα M βρίσκω το μέγεθος του y_k με ακέραια διαίρεση του 575 με το $M = 32$. Άρα, κάθε πλαίσιο ανάλυσης θα έχει τώρα 18 δείγματα.

Στο σχήμα 2.5 φαίνεται ένα πλαίσιο ανάλυσης (18 δειγμάτων) για $k = 10$.



Σχήμα 2.5

Βήμα 2.2: Κβαντοποίηση

Αρχικά κατασκευάζω τον προσαρμοζόμενο κβαντιστή. Πρώτο βήμα είναι να υπολογίσω το πλήθος των bits που θα χρησιμοποιηθούν για κάθε πλαίσιο ανάλυσης. Για να βρω το minimum $Tg(i)$ χρησιμοποιώ την `np.where()` απομονώνοντας τα i για τα οποία $f(i) \in [fk - F\pi/2M, fk + F\pi/2M]$. Στη συνέχεια υπολογίζω τα bits τα οποία είναι:

[9, 10, 9, 9, 9, 9, 8, 8, 8, 8, 7, 7, 7]

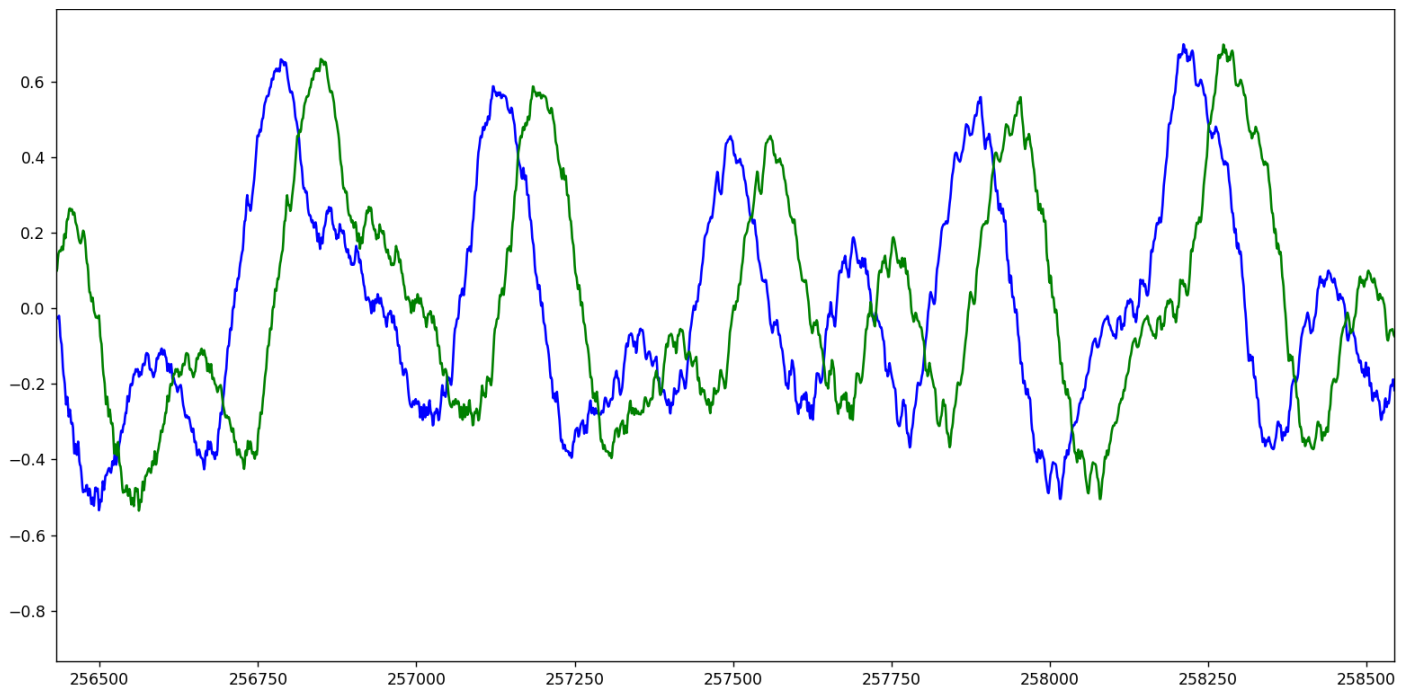
Όπως είναι φανερό τα bits κυμαίνονται μεταξύ 7 και 10 με μέσο όρο τα 9.3 bit ανά k. Τώρα, είμαι έτοιμος να κατασκευάσω έναν mid riser κβαντιστή με Δ το οποίο καθορίζεται από τα x_{min} , x_{max} κάθε πλαισίου ανάλυσης και από το εκάστοτε B_k . Επίσης, προσαρμόζεται και η θέση κβαντισμού σύμφωνα με την εκφώνηση.

Όσον αφορά στον 8-bit κβαντιστή η διαδικασία είναι πιο απλή. Τώρα, δεν απαιτείται ο υπολογισμός των bits αφού είναι 8 για κάθε παράθυρο ενώ το Δ είναι σταθερό και ίσο με $(1 - (-1)) / (2^8) = 1 / 2^7$

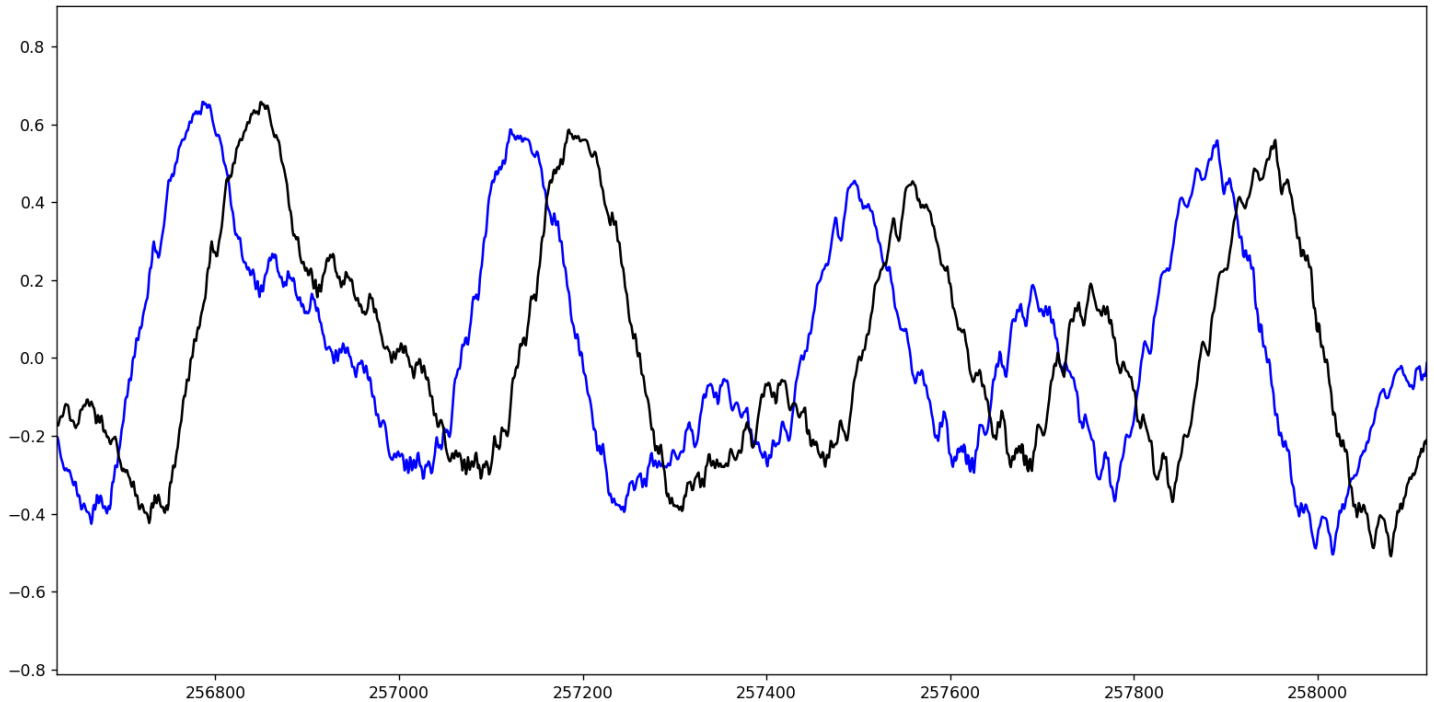
Λογικά, αναμένω το λάθος του 8-bit κβαντιστή να είναι μεγαλύτερο.

Βήμα 2.3: Σύνθεση

Ακολουθώντας τα βήματα της εκφώνησης κάνω υπερδειγματοληψία και στη συνέχεια συνελίσσω με τα φίλτρα σύνθεσης. Η υπερδειγματοληψία οδηγεί πάλι σε πλαίσια ανάλυσης 575 δειγμάτων ενώ η συνέλιξη αυξάνει τα δείγματα σε 638. Πραγματοποιώντας την τεχνική του overlap-add προσθέτω τα υπολειπόμενα δείγματα στο επόμενο παράθυρο και καταλήγω με πλαίσια ανάλυσης των 512 δειγμάτων. Στα σχήματα 2.6 και 2.7 φαίνονται τα τελικά ανακατασκευασμένα σήματα για τον προσαρμοζόμενο και τον 8-bit κβαντιστή αντίστοιχα (σε ένα εύρος δειγμάτων). Σε κάθε σχήμα απεικονίζεται και το αρχικό σήμα με μπλε χρώμα



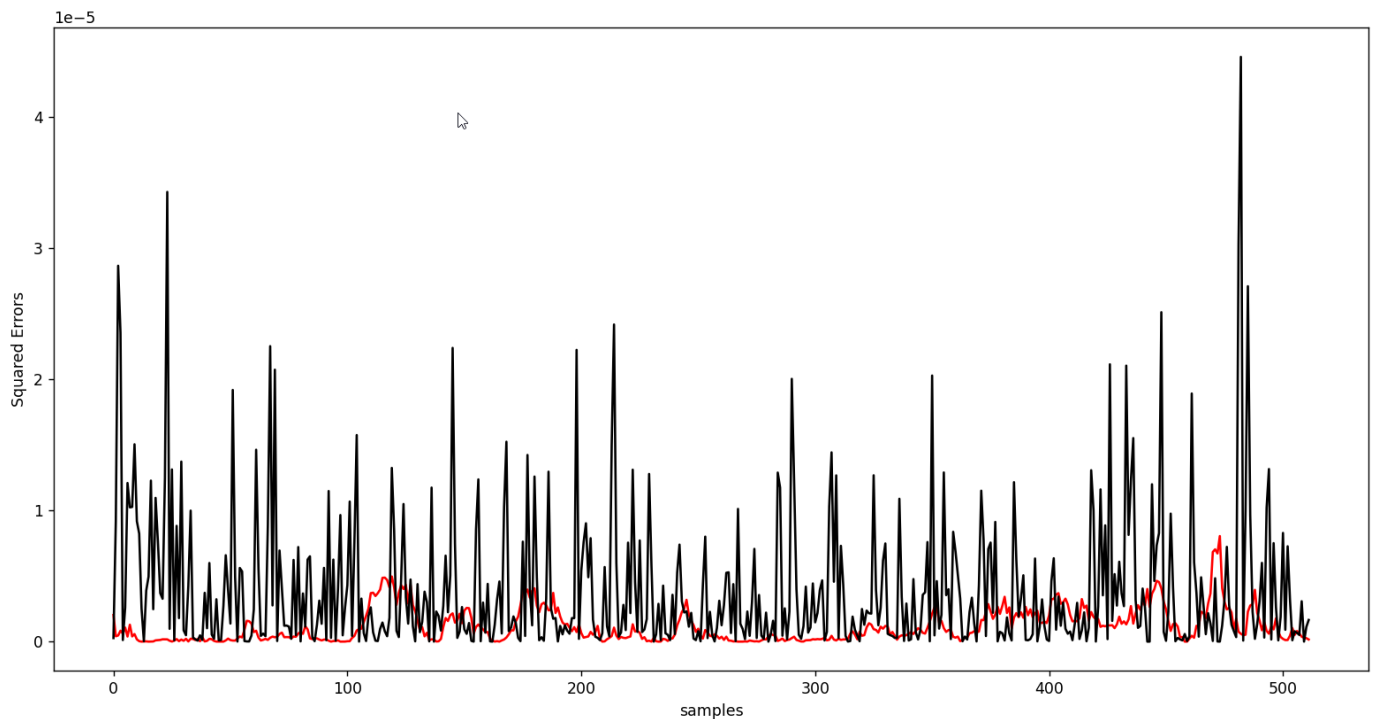
Σχήμα 2.6



Σχήμα 2.7

Από τα δύο σχήματα γίνεται εμφανές ότι το ανακατασκευασμένο σήμα έχει μία καθυστέρηση μεταξύ 62-64 δειγμάτων σε σχέση με το αρχικό σήμα μουσικής. Για να βρω την καθυστέρηση αυτή υπολογίζω για $\text{delay} = 62, 63, 64$ το μέσο τετραγωνικό σφάλμα. Παρατηρώ ότι έχω το ελάχιστο σφάλμα για $\text{delay} = 63$.

Υπολογίζω το μέσο τετραγωνικό σφάλμα για τις δύο περιπτώσεις. Για τον προσαρμοζόμενο κβαντιστή μετράω $1.1476621975963483 \times 10^{-6}$, ενώ για τον 8-bit $3.703135639310995 \times 10^{-6}$. Παρατηρώ ότι το μέσο τετραγωνικό σφάλμα είναι μικρότερο για τον προσαρμοζόμενο κβαντιστή, όπως ανέμενα. Στο σχήμα 2.8 φαίνονται τα δύο τετραγωνικά σφάλματα.



Σχήμα 2.8

Με κόκκινο χρώμα απεικονίζεται ο προσαρμοζόμενος κβαντιστής και με μαύρο ο 8-bit. Από το σχήμα φαίνεται ότι ο προσαρμοζόμενος κβαντιστής μειώνει πολύ το σφάλμα (μία τάξη μεγέθους μικρότερο).

Ωστόσο, ο προσαρμοζόμενος κβαντιστής μειώνει το ποσοστό συμπίεσης. Πιο συγκεκριμένα, όπως αναφέρεται και στην εκφώνηση στον αποκωδικοποιητή στέλνεται και η πρώτη στάθμη της κβάντισης και το Δ . Για τον 8-bit κβαντιστή τα δύο αυτά μεγέθη είναι σταθερά οπότε αρκεί να σταλούν μόνο μία φορά και το overhead θα είναι 32 bits (16 + 16). Για τον προσαρμοζόμενο κβαντιστή θα σταλούν μία φορά για κάθε φίλτρο και άρα το overhead θα είναι $32 * 32 = 1024$ bits.

Για τον υπολογισμό του ποσοστού συμπίεσης για κάθε κβαντιστή ακολουθώ την εξής διαδικασία:

1. Υπολογισμός αρχικού αριθμού bits = $\text{bits_per_sample} * \text{samples}$ για το σήμα μουσικής, με $\text{bits_per_sample} = 16$.
2. Για κάθε κβαντιστή υπολογισμός του τελικού αριθμού bits. Για τον 8-bit είναι $8 * \text{samples}$ ενώ για τον adaptive $\text{bits_for_sample} * \text{samples}$.
3. Πρόσθεση του overhead για κάθε κβαντιστή
4. $\Pi = (1 - \text{τελικός αριθμός bits} / \text{αρχικό αριθμό bits}) * 100\%$

Έτσι, καταλήγω σε ποσοστό συμπίεσης ίσο με 35% για τον προσαρμοζόμενο κβαντιστή (0.34471476996775985) και 44% για τον 8-bit κβαντιστή (0.43715822538269644).

Άρα, η τελική επιλογή του κβαντιστή εξαρτάται από το συνδυασμό του ζητούμενου σφάλματος και ποσοστού συμπίεσης. Για μικρότερο σφάλμα θα προτιμήσω τον προσαρμοζόμενο κβαντιστή ενώ για μικρότερο ποσοστό συμπίεσης τον 8-bit κβαντιστή.